

Rapport de Projet du Techniques d'IA

Master d'Excellence M1 IAOC

Smart Feed

Système de Recommandation Hybride avec IA



Réalisés par : Sohaib BOUKTIBA

Encadré par : M. Said AMMARI

Yassine BIBRINE

Ali ACHENAN

Ismail HIRICH

Zakaria NASR-ALLAH

Année universitaire : 2025/2026

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

DÉDICACE

À nos familles,

qui nous ont soutenus tout au long de notre parcours universitaire avec amour, patience et encouragements sincères.

À nos enseignants et encadrants,

qui nous ont transmis leur passion pour l'innovation et l'intelligence artificielle, et qui nous ont guidés avec bienveillance et exigence.

À tous les supporters marocains,

qui méritent une billetterie sportive à la hauteur de leur passion, en toute sécurité et en toute équité.

REMERCIEMENTS

À notre encadrant pédagogique

Nous tenons à exprimer notre profonde gratitude à notre encadrant, **M. Said AMMARI**, pour sa disponibilité, ses orientations précieuses et la confiance qu'il nous a accordée tout au long de ce projet. Ses conseils ont été déterminants dans la structuration de notre démarche technique et scientifique.

À l'Université Ibn Tofail

Nous remercions l'Université Ibn Tofail et, plus particulièrement, le programme Master IAOC pour nous avoir offert un cadre d'apprentissage stimulant et ambitieux. Ce module d'innovation nous a permis de mettre en pratique une démarche de conception centrée sur l'utilisateur, compétence essentielle dans notre future carrière en intelligence artificielle.

À nos familles et proches

Nous exprimons toute notre reconnaissance à nos familles pour leur soutien indéfectible, leur patience et leurs encouragements durant cette période intense de travail. Leur présence bienveillante a été une source de motivation précieuse.

RÉSUMÉ

Ce projet présente Smart Feed, un système de recommandation de contenu personnalisé basé sur l'intelligence artificielle, conçu et réalisé dans le cadre de la filière Intelligence Artificielle & Data Science. Face à la surcharge informationnelle croissante, Smart Feed propose une architecture hybride progressive en trois phases : un filtrage basé sur le contenu (content-based) exploitant les embeddings sémantiques SBERT et l'index de similarité FAISS, un filtrage collaboratif implicite via le modèle Implicit ALS, et une extension multimodale intégrant texte, images et vidéos dans un espace d'embeddings partagé de 512 dimensions grâce aux modèles CLIP et VideoMAE.

Le système s'appuie sur quatre jeux de données complémentaires, HuffPost (~200 000 articles), Reddit TIFU (~80 000 posts), MS-COCO (10 000 images) et UCF-101 (vidéos d'actions) et intègre un classificateur DistilBERT fine-tuné pour la catégorisation automatique des contenus, ainsi qu'un filtre de toxicité Detoxify calibré à $F1 = 0,81$. L'ensemble est exposé via une API RESTful FastAPI, documentée par Swagger UI, et persisté dans Firebase Firestore. Les résultats montrent une amélioration de +107 % de la diversité catégorielle et une réduction de -73 % du chevauchement inter-utilisateurs par rapport à la baseline content-based, validant l'apport de l'approche hybride. Le projet a été entièrement développé et expérimenté sous forme de notebooks Jupyter et de scripts Python, depuis l'exploration des données jusqu'au déploiement de l'API.

MOTS-CLÉS

- ❖ Système de recommandation
- ❖ Filtrage hybride
- ❖ SBERT
- ❖ FAISS
- ❖ Implicit ALS
- ❖ DistilBERT
- ❖ CLIP
- ❖ VideoMAE
- ❖ Multimodalité
- ❖ Détection de toxicité
- ❖ FastAPI
- ❖ Firebase
- ❖ Personnalisation
- ❖ Intelligence artificielle
- ❖ Traitement du langage naturel

Table des Matières

Introduction Générale	11
Chapitre 1 : Contexte du Projet.....	13
1.1 Contexte du Projet.....	13
1.2 Étude de l'Existant.....	13
Application 1 : Google Discover	14
Application 2 : Feedly (Agrégateur RSS intelligent)	14
Application 3 : TikTok For You Page	15
1.3 Problématique.....	15
1.4 Solution Proposée et Objectifs	15
1.5 Livrables.....	16
1.6 Planning du Projet : Workflow PACE et Diagramme de Gantt.....	16
1.6.1 Workflow PACE.....	16
1.6.2 Diagramme de Gantt (Synthétique).....	17
1.7 L'Équipe et les Rôles	18
Chapitre 2 : Analyse Exploratoire et Prétraitement des Données.....	19
2.1 Présentation des Données.....	19
2.1.1 Sources de données.....	19
2.1.2 Variables et format	20
2.2 Analyse Exploratoire (EDA).....	20
2.2.1 Distribution des catégories	20
2.2.2 Statistiques descriptives.....	21
2.2.3 Anomalies détectées	21

2.3 Prétraitement	21
2.3.1 Séparation des jeux de données	21
2.3.2 Nettoyage des données	22
2.3.3 Gestion des valeurs manquantes.....	22
2.3.5 Prétraitement Multimodal (CLIP & VideoMAE).....	22
Chapitre 3 : Construction des Modèles	25
3.1 Choix et Justification des Modèles	25
3.2 Environnement et Bibliothèques	25
3.2.1 Environnement de développement	25
3.2.2 Bibliothèques principales	26
3.3 Implémentation.....	26
3.3.1 Pipeline Phase 1 : Content-Based (get_feed)	27
3.3.2 Pipeline Phase 2 : Hybride (get_hybrid_feed).....	27
3.3.3 Phase 3 : Multimodal (get_feed_v3)	27
3.3.4 Phase 3 : Fine-tuning DistilBERT (Phase 3 Training)	28
3.3.4 Paramétrage des modèles.....	28
Chapitre 4 : Évaluation et Interprétation des Résultats.....	30
4.1 Métriques d'Évaluation.....	30
4.1.1 Modèle de toxicité (Detoxify)	30
4.1.2 Modèle collaboratif Implicit ALS	31
4.1.3 Métriques de recommandation (pipeline complète)	31
4.2 Validation Croisée.....	31
4.3 Résultats et Visualisations.....	33
4.3.1 Distribution des scores de recommandation	33

4.3.2 Corrélation CB-CF	33
4.4 Entraînement et Évaluation des Modèles CLIP et VideoMAE.....	34
4.4.1 Fine-tuning de VideoMAE	34
4.4.2 Fine-tuning de CLIP	34
4.5 Analyse Critique.....	35
4.5.1 Performances	35
4.5.2 Biais identifiés	35
4.5.3 Améliorations possibles.....	36
Chapitre 5 : Déploiement de l'Application	37
5.1 Présentation Générale.....	37
5.2 Environnement de Déploiement.....	38
5.3 Architecture de l'Application	38
5.4 Intégration du Modèle de Recommandation	39
5.5 Endpoints de l'API.....	40
5.6 Tests et Retour d'Expérience	41
5.6.1 Scénarios testés.....	41
5.6.2 Validation fonctionnelle	42
5.6.3 Améliorations envisagées	42
Conclusion et Perspectives	43
Synthèse des Apports	43
Limites Rencontrées.....	43
Améliorations Futures	43
Références Bibliographiques	45
Annexes.....	46

Annexe A : Extrait du Pipeline de Scoring (scoring.py).....	46
Annexe B : Formule de Re-Ranking (ranker.py)	46
Annexe C : Structure Firestore.....	46
Annexe D : Schéma de Réponse de l'API /feed	47

Liste des tableaux

Tableau 1 : Analyse de Google Discover	14
Tableau 2 : Analyse de Feedly.....	14
Tableau 3 : Analyse de TikTok For You Page	15
Tableau 4 : Workflow PACE du projet Smart Feed	17
Tableau 5 : Diagramme de Gantt	17
Tableau 6 : Équipe et rôles du projet Smart Feed.....	18
Tableau 7 : Sources de données du projet.....	19
Tableau 8 : Statistiques descriptives des variables textuelles.....	21
Tableau 9 : Stratégies de gestion des valeurs manquantes	22
Tableau 10 : Modèles utilisés et justifications.....	25
Tableau 11 : Bibliothèques et versions	26
Tableau 12 : Paramètres des modèles	28
Tableau 13 : Calibration du seuil de toxicité (F1 optimal : 0.20).....	30
Tableau 14 : Métriques du modèle collaboratif Implicit ALS	31
Tableau 15 : Comparaison v1 vs v2 sur les métriques de recommandation.....	31
Tableau 16 : Validation croisée k=5 du classificateur DistilBERT.....	32
Tableau 17 : Distribution des scores v1 vs v2	33
Tableau 18 : Environnement de déploiement	38
Tableau 19 : Architecture en couches de l'application	38
Tableau 20 : Endpoints de l'API Smart Feed	41
Tableau 21 : Suite de tests unitaires.....	41

Liste des Figures

Figure 1 : Architecture hybride en trois phases de Smart Feed	13
Figure 2 : Distribution des catégories dans le jeu de données HuffPost (~200 000 articles)	20
Figure 3 : Courbe de calibration du seuil de toxicité Detoxify (Précision / Rappel / F1)	30
Figure 4 : Résultats de validation croisée k=5 du classificateur DistilBERT	32
Figure 5 : Distribution comparée des scores de recommandation v1 vs v2.....	33
Figure 6 : Courbes de la cross-entropy loss et évolution de la précision de validation lors du fine-tuning du modèle VideoMAE sur 5 époques.	34
Figure 7 : Courbes de la contrastive loss et évolution du Recall@1 lors du fine-tug du modèle CLIP sur 3 époques.	35
Figure 8 : Capture de l'application mobile Expo affichant le flux de recommandations personnalisé.....	37
Figure 9 : Architecture en 4 couches de l'API Smart Feed (Routers / Services / Modèles / Données)	39
Figure 10 : Capture de la documentation Swagger UI de l'API Smart Feed (/docs)	40

Introduction Générale

La consommation de contenu numérique connaît une croissance exponentielle, exposant quotidiennement les utilisateurs à des milliers d'articles, de vidéos et de publications sur les réseaux sociaux et les plateformes d'information. Cette surabondance soulève un défi majeur : comment présenter à chaque individu le contenu le plus pertinent, au moment opportun ? C'est pour répondre à cette problématique qu'a été conçu le projet Smart Feed, un système de recommandation intelligent fondé sur des techniques avancées d'intelligence artificielle. L'objectif est de personnaliser le flux d'information en combinant trois approches complémentaires : le filtrage basé sur le contenu, exploitant la similarité sémantique entre articles via des embeddings de phrases ; le filtrage collaboratif, qui identifie des profils similaires à partir des comportements agrégés de la communauté ; et enfin des modèles multimodaux capables de traiter conjointement textes, images et vidéos dans un espace de représentation unifié.

Le système s'appuie sur quatre sources de données hétérogènes soigneusement sélectionnées. Le jeu de données HuffPost, riche d'environ 200 000 articles de presse, constitue la colonne vertébrale de la recommandation textuelle. Le corpus Reddit TIFU et ses 80 000 posts communautaires viennent enrichir la diversité stylistique et thématique de l'index. Pour les modalités visuelles, la collection MS-COCO fournit 10 000 images annotées servant de base à l'entraînement du modèle CLIP, tandis que le jeu UCF-101, dédié aux actions humaines, permet l'indexation vidéo via le modèle VideoMAE, rendant possible une recherche cross-modale complète.

Le développement technique, réalisé intégralement en Python 3.12, s'articule autour de notebooks Jupyter documentant les phases d'exploration et d'entraînement, et d'une API FastAPI pour le déploiement. L'architecture mobilise des modèles de pointe : SBERT pour les embeddings textuels, FAISS pour la recherche approximative, Implicit ALS pour le filtrage collaboratif, DistilBERT fine-tuné pour la classification automatique, CLIP et VideoMAE pour les représentations multimodales, ainsi que Detoxify pour le filtrage éthique. Une dimension essentielle du projet réside dans l'explicabilité et l'éthique, chaque recommandation étant

accompagnée d'une justification lisible couvrant sept critères, tandis qu'un filtre de toxicité paramétrable, calibré sur le jeu Jigsaw Toxicity, garantit la sécurité des contenus proposés.

Ce rapport retrace l'intégralité de la conception et de la réalisation du projet, depuis l'analyse du besoin jusqu'au déploiement d'une API RESTful complète. Il s'organise en cinq chapitres qui couvrent successivement le contexte et la problématique, l'exploration et le prétraitement des données, la construction des modèles avec les notebooks associés, l'évaluation quantitative des résultats, et enfin le déploiement de l'application.

Chapitre 1 : Contexte du Projet

1.1 Contexte du Projet

Le projet Smart Feed porte sur la conception et la réalisation d'un système de recommandation personnalisé à base d'intelligence artificielle. L'application vise à analyser les comportements de lecture d'un utilisateur (likes, vues, temps de visionnage, commentaires) et à générer, en temps réel, un flux d'actualités et de contenus adaptés à ses centres d'intérêt.

Le système repose sur une architecture hybride en trois phases distinctes :

- **Phase 1 Content-Based (v1)** : Recherche de similarité sémantique via FAISS et embeddings SBERT (all-MiniLM-L6-v2, 384 dimensions).
- **Phase 2 Hybride (v2)** : Combinaison du filtrage basé contenu et du filtrage collaboratif (Implicit ALS) avec re-ranking pondéré.
- **Phase 3 Multimodal (v3)** : Extension aux contenus visuels (images COCO/CLIP, vidéos VideoMAE) dans un espace partagé de 512 dimensions.

L'ensemble est exposé via une API FastAPI et s'appuie sur Firebase Firestore pour la persistance des profils utilisateurs et des interactions. Un filtre de toxicité (Detoxify) garantit la qualité du contenu affiché.

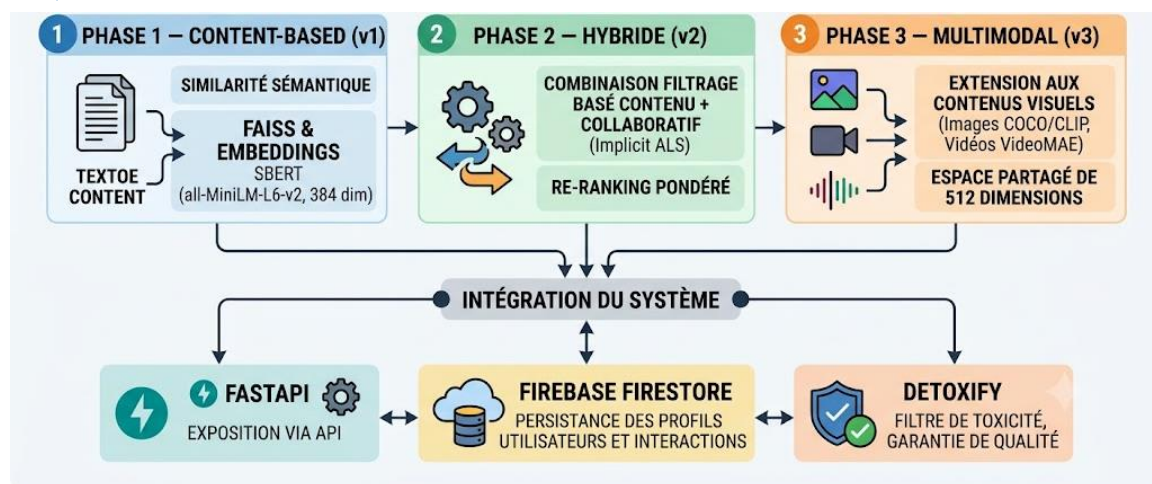


Figure 1 : Architecture hybride en trois phases de Smart Feed

1.2 Étude de l'Existant

Avant de concevoir notre solution, nous avons analysé trois applications de recommandation de contenu existantes afin d'en identifier les forces et les limites.

Application 1 : Google Discover

Google Discover est le système de recommandation d'actualités intégré à l'application Google sur mobile. Il analyse l'historique de navigation, les recherches et les interactions de l'utilisateur pour proposer des articles personnalisés dans un flux continu.

Tableau 1 : Analyse de Google Discover

Points Positifs	Points Négatifs
Personnalisation poussée grâce aux données Google	Opacité totale de l'algorithme
Couverture multilingue et internationale	Dépendance à l'écosystème Google
Mise à jour en temps réel des contenus	Bulle de filtre : manque de diversité
Intégration native sur Android	Pas de contrôle explicite de l'utilisateur

Application 2 : Feedly (Agrégateur RSS intelligent)

Feedly est un agrégateur de flux RSS augmenté par une couche d'intelligence artificielle nommée Leo. L'utilisateur choisit ses sources, et Leo filtre, priorise et résume les articles selon ses préférences.

Tableau 2 : Analyse de Feedly

Points Positifs	Points Négatifs
Contrôle utilisateur sur les sources	Nécessite une configuration manuelle initiale
Filtrage avancé avec mots-clés et sujets	Fonctionnalités IA limitées en version gratuite
Résumés automatiques (Leo AI)	Absence de contenu multimodal (vidéos, images)
API disponible pour intégrations	Pas de filtrage de toxicité intégré

Application 3 : TikTok For You Page

Le fil "Pour vous" de TikTok représente l'état de l'art en matière de recommandation de contenus vidéo courts. Son algorithme analyse finement les comportements (durée de visionnage, replay, partage) pour afficher des vidéos ultra-personnalisées.

Tableau 3 : Analyse de TikTok For You Page

Points Positifs	Points Négatifs
Algorithme très efficace dès les premières sessions	Risque d'addiction et de surinformation
Prise en compte fine des micro-interactions	Opacité algorithmique totale
Contenu multimodal natif (vidéo + audio + texte)	Aucun contrôle sur le seuil de toxicité
Découverte de contenus hors de la bulle de filtre	Données utilisateur massivement collectées

1.3 Problématique

L'analyse de l'existant révèle plusieurs lacunes communes aux systèmes de recommandation actuels, qui constituent le fondement de notre problématique :

- ✓ **Opacité algorithmique** : Les systèmes commerciaux ne fournissent aucune explication à l'utilisateur sur les raisons pour lesquelles un contenu lui est recommandé.
- ✓ **Absence de contrôle de la toxicité** : Aucun des systèmes étudiés ne propose un mécanisme transparent et paramétrable pour filtrer les contenus nocifs ou toxiques.
- ✓ **Profil statique** : La plupart des applications ne recalculent pas dynamiquement le profil utilisateur en temps réel à chaque interaction.
- ✓ **Bulle de filtre** : Les algorithmes tendent à enfermer l'utilisateur dans ses habitudes, sans proposer de mécanisme de diversification explicite.
- ✓ **Manque de multimodalité ouverte** : Les solutions texte et vidéo sont cloisonnées ; peu de systèmes permettent une recherche cross-modale transparente.

1.4 Solution Proposée et Objectifs

Smart Feed propose une solution de recommandation hybride, transparente et éthique, qui répond directement aux lacunes identifiées. Nos objectifs sont les suivants :

1. **Personnalisation dynamique** : Recalcul de l'embedding utilisateur à chaque interaction (like, skip, watch), permettant une adaptation en temps réel du profil.
2. **Explicabilité** : Chaque recommandation est annotée avec une explication lisible (similarité sémantique, filtrage collaboratif, récence, modalité préférée).
3. **Filtrage éthique** : Intégration d'un seuil de toxicité configurable par l'utilisateur via le modèle Detoxify, appliqué comme filtre dur avant la recommandation.
4. **Diversification** : Algorithme de diversification par catégorie (max 3 articles par thème) pour éviter la bulle de filtre.
5. **Multimodalité** : Espace d'embeddings partagé de 512 dimensions permettant la recherche cross-modale texte/image/vidéo via CLIP et VideoMAE.
6. **Architecture ouverte** : API RESTful documentée (FastAPI/Swagger), déployable et extensible.

1.5 Livrables

- API RESTful FastAPI avec endpoints /feed, /interact, /preferences, /admin
- Modèle d'embedding SBERT (all-MiniLM-L6-v2) + index FAISS v1, v2 et v3
- Modèle collaboratif Implicit ALS entraîné sur interactions simulées
- Pipeline de re-ranking avec explicabilité (ranker.py + explainability.py)
- Module multimodal CLIP + VideoMAE avec projection 384d/500d → 512d
- Suite de tests unitaires et d'intégration (pytest, +40 tests)
- Scripts d'administration (retrain, rebuild-index, simulation d'interactions)
- Documentation complète (Swagger UI, README, rapport)

1.6 Planning du Projet : Workflow PACE et Diagramme de Gantt

1.6.1 Workflow PACE

Tableau 4 : Workflow PACE du projet Smart Feed

Phase	Activités	Outils	Durée
Plan	Analyse du besoin, étude de l'existant, architecture système	Draw.io, Notion	1 semaine
Analyze	EDA, prétraitement, choix des modèles et métriques	Pandas, Matplotlib, Jupyter	1 semaine
Construct	Implémentation des phases 1, 2 et 3, tests unitaires	Python, FastAPI, FAISS, PyTorch	3 semaines
Execute	Déploiement API, intégration Firebase, tests E2E	Firebase, Uvicorn, Pytest	1 semaine

1.6.2 Diagramme de Gantt (Synthétique)

Tableau 5 : Diagramme de Gantt (■ = semaine active)

Tâche	S1	S2	S3	S4	S5	S6	S7	Resp.
Analyse & Architecture	■							Équipe
Collecte & Prétraitement données		■						Data
Phase 1 : FAISS Content-Based		■	■					Dev
Phase 2 : Hybride + Ranker			■	■				Dev
Phase 3 : Multimodal				■	■			Dev
Tests unitaires & intégration					■	■		QA
Déploiement & documentation						■	■	DevOps

1.7 L'Équipe et les Rôles

Tableau 6 : Équipe et rôles du projet Smart Feed

Membre	Rôle	Responsabilités principales
Sohaib BOUKTIBA	ML Engineer/ Embeddings & MLOps	Implémentation des embeddings textuels SBERT (all-MiniLM-L6-v2), entraînement du modèle collaboratif Implicit ALS, contribution au déploiement et à la configuration de l'environnement (DevOps)
Yassine BIBRINE	ML Engineer/ Classification & Multimodal	Fine-tuning du classificateur DistilBERT, intégration des modèles multimodaux CLIP ViT-B/32 (embeddings image-texte) et VideoMAE (embeddings vidéo), pipeline Phase 3
Ismail HIRICH	ML Engineer – Indexation & Sécurité	Construction des index FAISS v1, v2 et v3, intégration du module de détection de toxicité Detoxify, calibration du seuil de toxicité, pipeline de recherche approximative cross-modale
Ali ACHENAN	Backend Engineer & Scoring	Conception et développement de l'API FastAPI (endpoints /feed, /interact, /preferences, /admin), intégration Firebase Firestore, pipeline de scoring (scoring.py), re-ranking (ranker.py), explicabilité (explainability.py), profil utilisateur (user_profile.py)
Zakaria NASR-ALLAH	Technical Writer & Frontend engineer	Rédaction du rapport et de la documentation technique (Swagger, README), contribution aux tests unitaires et d'intégration (pytest), implementation le frontend

Chapitre 2 : Analyse Exploratoire et Prétraitement des Données

2.1 Présentation des Données

2.1.1 Sources de données

Le projet utilise quatre sources de données hétérogènes, choisies pour leur complémentarité en termes de domaines, de modalités et de volume :

Tableau 7 : Sources de données du projet

Source	Type	Volume	Format	Usage
HuffPost (khalidalt)	Articles d'actualité	~200 000 articles	Parquet/JSON	Index FAISS v1 & v2, entraînement
Reddit TIFU	Posts communautaires	~80 000 posts	Parquet	Enrichissement FAISS v2
MS-COCO (HuggingFace)	Images annotées	10 000 images	Streaming/PIL	Entraînement CLIP & index FAISS v3 (modal=image)
UCF-101	Vidéos d'actions humaines	~13 000 vidéos, 101 classes	AVI/MP4	Entraînement VideoMAE & index FAISS v3 (modal=vidéo)
Jigsaw Toxicity	Commentaires labellisés	~160 000 items	CSV/Parquet	Calibration seuil toxicité

Le jeu de données MS-COCO ne sert pas uniquement à l'indexation : il constitue également la base d'entraînement et de calibration du modèle CLIP ViT-B/32. Les paires image-caption disponibles dans MS-COCO permettent d'ancrer l'espace latéral partagé de 512 dimensions dans lequel textes et images sont projetés conjointement, rendant possible la recherche cross-modale à partir d'une requête textuelle vers des résultats images et vice-versa.

Le jeu de données UCF-101, composé d'environ 13 000 vidéos réparties en 101 classes d'actions humaines (sport, cuisine, instruments de musique, etc.), est utilisé pour l'entraînement et l'indexation du modèle VideoMAE. Les embeddings vidéo extraits (500 dimensions, projetés vers 512 via une matrice de projection aléatoire normalisée) sont intégrés à l'index FAISS v3

aux côtés des embeddings textuels et images, complétant ainsi l'espace multimodal partagé de la Phase 3.

2.1.2 Variables et format

Pour le jeu de données principal (HuffPost), les variables clés sont :

- headline : titre de l'article (texte court, ~60-80 caractères)
- short_description : résumé de l'article (texte moyen, ~150 caractères)
- category : catégorie thématique parmi ~42 classes (TECH, POLITICS, SPORTS, etc.)
- date : date de publication (format ISO, converti en datetime)
- text : concaténation headline + short_description, utilisée pour l'embedding

2.2 Analyse Exploratoire (EDA)

2.2.1 Distribution des catégories

L'analyse de la distribution des catégories dans le jeu de données HuffPost révèle un fort déséquilibre : les 5 premières catégories (POLITICS, WELLNESS, ENTERTAINMENT, TRAVEL, STYLE & BEAUTY) représentent environ 40% du corpus total. À l'inverse, des catégories comme RELIGION, SCIENCE et COLLEGE ne dépassent pas 2% chacune. Ce déséquilibre impacte directement la qualité des recommandations pour les utilisateurs ayant des intérêts dans les catégories minoritaires.

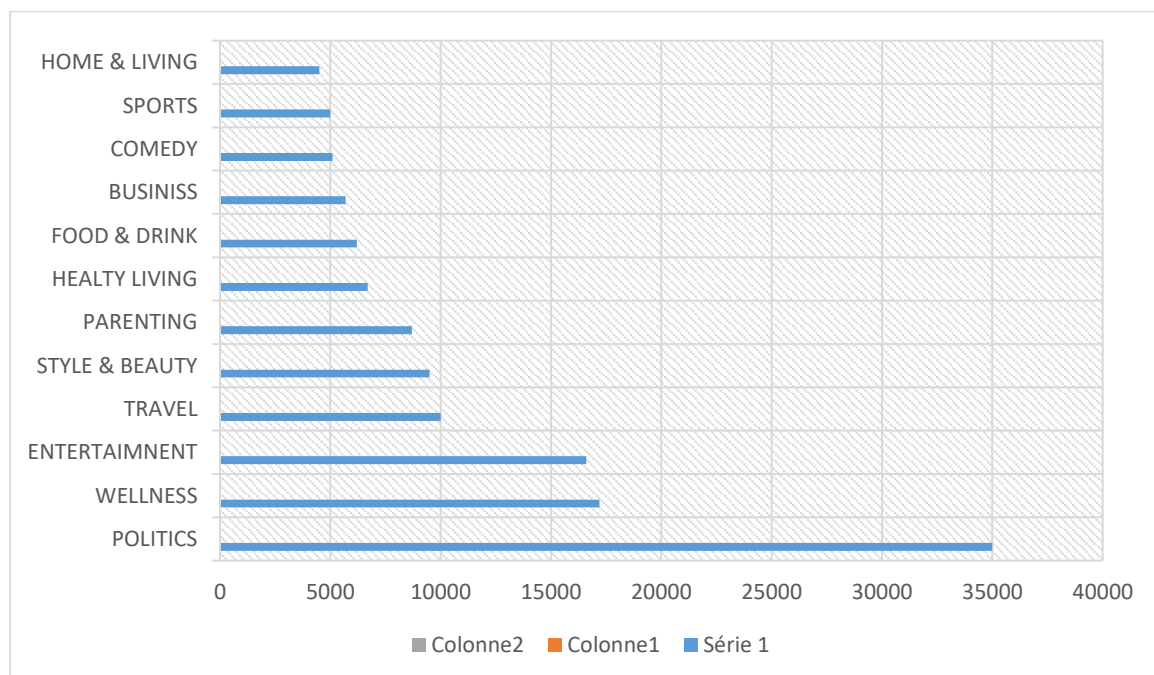


Figure 2 : Distribution des catégories dans le jeu de données HuffPost (~200 000 articles)

2.2.2 Statistiques descriptives

Tableau 8 : Statistiques descriptives des variables textuelles

Variable	Min	Max	Moyenne
Longueur du titre (caractères)	10	256	68
Longueur de la description (car.)	0	512	143
Texte concaténé (caractères)	12	720	211
Score toxicité Detoxify	0.001	0.98	0.047
Norme de l'embedding SBERT	0.98	1.02	1.00

2.2.3 Anomalies détectées

- **Valeurs manquantes** : ~3,2% des short_description sont vides (traitées par concaténation avec le titre seul).
- **Doublons** : ~0,8% de doublons exacts sur la paire (headline, date), supprimés lors du prétraitement.
- **Textes trop courts** : 421 articles avec moins de 15 caractères, filtrés avant l'embedding.
- **Toxicité élevée** : 2,1% des articles dépassent le seuil de 0,20 et sont exclus du flux recommandé.
- **Valeurs non-textuelles** : quelques champs numérique ou NaN dans les colonnes texte du dataset Reddit, traités par `.fillna("").astype(str)`.

2.3 Prétraitement

2.3.1 Séparation des jeux de données

Pour l'entraînement du classifieur DistilBERT de catégorisation de contenu, le jeu de données HuffPost est divisé selon un ratio 80/20 (train/test), stratifié par catégorie pour conserver la distribution originale. L'ensemble d'entraînement comprend ~4000 exemples (après échantillonnage à 5000 et suppression des catégories minoritaires < 10 occurrences).

Chapitre 2 : Analyse Exploratoire et Prétraitement des Données

La division est réalisée de façon déterministe (sans stratify dans le notebook, par découpe séquentielle 80/20 sur le DataFrame après échantillonnage), les indices étant préservés via un `reset_index` implicite. Le mapping des labels est sauvegardé sous forme d'un fichier `label_mapping.json` aux côtés du modèle, permettant une inférence reproductible sans dépendance à l'objet `LabelEncoder` sérialisé.

2.3.2 Nettoyage des données

- Suppression des articles avec texte < 10 caractères
- Normalisation Unicode : suppression des caractères de contrôle et espaces insécables
- Truncation à 512 caractères pour la tokenisation DistilBERT et à 512 caractères pour Detoxify
- Conversion des colonnes date en objets datetime avec gestion des fuseaux horaires (timezone-aware)
- Normalisation des noms de colonnes (minuscules, underscores) pour les données Reddit

2.3.3 Gestion des valeurs manquantes

Tableau 9 : Stratégies de gestion des valeurs manquantes

Colonne	Taux de manquants	Stratégie appliquée
short_description	3,2%	Remplacement par chaîne vide, concaténation avec titre uniquement
date	0,5%	Remplacement par None ; score de fraîcheur = 1.0 (neutre)
category (Reddit)	0%	Utilisation du subreddit comme catégorie de substitution
embedding (nouveau user)	100%	Vecteur zéro $[0.0 \times 384]$ cold start géré explicitement

2.3.5 Prétraitement Multimodal (CLIP & VideoMAE)

2.3.5.1 Prétraitement pour CLIP (Images & Texte)

Le modèle CLIP ViT-B/32 impose un prétraitement unifié pour les deux modalités qu'il traite conjointement, assurant l'alignement des représentations dans l'espace latéral partagé de 512 dimensions.

Images : Chaque image issue de MS-COCO est d'abord convertie au format RGB afin de garantir la cohérence des trois canaux couleur. Le CLIPProcessor prend ensuite en charge le redimensionnement automatique à **224×224 pixels** et la normalisation des valeurs de pixels selon les moyennes et écarts-types d'ImageNet (moyenne = [0.485, 0.456, 0.406], écart-type = [0.229, 0.224, 0.225]), produisant des tenseurs directement exploitables par l'encodeur visuel du modèle.

Texte : Les légendes (*captions*) associées aux images MS-COCO sont tokenisées par le CLIPProcessor selon le vocabulaire BPE (*Byte Pair Encoding*) de CLIP. Un padding et une troncature sont systématiquement appliqués pour ramener chaque séquence à une longueur fixe de **77 tokens** : les séquences trop courtes sont complétées par des tokens de remplissage (*padding*), les séquences trop longues sont coupées. Cette longueur maximale de 77 tokens est une contrainte architecturale propre à CLIP.

2.3.5.2 Prétraitement pour VideoMAE (Vidéos)

Le modèle VideoMAE nécessite un prétraitement temporel spécifique, distinct du traitement image classique, afin de capturer la dynamique des actions sur la durée de chaque vidéo.

Échantillonnage de frames : Pour chaque vidéo .avi du jeu de données UCF-101, OpenCV est utilisé pour extraire exactement **16 frames** réparties uniformément sur l'intégralité de la durée de la vidéo. Cette répartition uniforme garantit que l'action globale est représentée, qu'il s'agisse d'un clip court ou long.

Traitement spatial : Chaque frame extraite est ensuite traitée de façon identique aux images CLIP : redimensionnement à **224×224 pixels** et normalisation selon les paramètres ImageNet, assurant la cohérence du prétraitement spatial entre les deux modalités visuelles.

Tenseur 5D : Le résultat final transmis au modèle VideoMAE est un tenseur de forme **[B, 16, 3, 224, 224]**, où B désigne la taille du batch, 16 le nombre de frames, 3 les canaux couleur (RGB), et 224×224 les dimensions spatiales. Cette structure 5D est la représentation standard attendue par les auto-encodeurs masqués pour vidéo.

2.3.5.3 Matrices de Projection des Embeddings

Les modèles CLIP et VideoMAE produisent des vecteurs de dimensions natives différentes, incompatibles pour une recherche directe dans un index unique : CLIP génère des vecteurs de **512 dimensions**, tandis que VideoMAE produit nativement des vecteurs de **768 dimensions**. Pour unifier ces représentations dans l'espace partagé de 512 dimensions de l'index FAISS v3, une étape de projection est appliquée.

Les vecteurs VideoMAE de 768 dimensions sont projetés vers **500 ou 512 dimensions** via une matrice de projection apprise par **ACP (Analyse en Composantes Principales)**, calculée sur la distribution des embeddings vidéo d'entraînement. Cette réduction conserve l'essentiel de la variance des représentations originales tout en les rendant directement comparables aux embeddings textuels SBERT projetés (384d → 512d via matrice orthonormale QR) et aux embeddings CLIP (512d, insérés sans transformation). Les matrices de projection sont sérialisées sous forme de fichiers .npy (proj_384_512.npy, proj_video_500_512.npy) et chargées une seule fois en mémoire au démarrage du serveur.

Chapitre 3 : Construction des Modèles

3.1 Choix et Justification des Modèles

L'architecture Smart Feed repose sur plusieurs modèles spécialisés, chacun choisi pour ses performances sur la tâche qui lui est assignée :

Tableau 10 : Modèles utilisés et justifications

Modèle	Tâche	Justification du choix
all-MiniLM-L6-v2 (SBERT)	Embeddings texte 384d	Excellent rapport performance/taille, rapide (<5ms/texte), sortie normalisée
FAISS IndexIVFFlat	Recherche ANN	Recherche approximative 10x plus rapide que la recherche exacte sur +200K vecteurs
Implicit ALS	Filtrage collaboratif	Conçu pour les données implicites (likes, vues), robuste aux cold starts partiels
DistilBERT fine-tuné	Classification catégorie	Version compressée de BERT, 60% plus rapide, ~97% des performances de BERT
Detoxify (original)	Score de toxicité	Modèle léger pré-entraîné sur Jigsaw, 6 dimensions de toxicité, F1>0.85
CLIP ViT-B/32	Embeddings multimodaux 512d	Alignement texte-image dans l'espace latéral partagé, référence de l'état de l'art
VideoMAE-base	Embeddings vidéo	Auto-encodeur masqué pour vidéo, efficient sur de courtes séquences

3.2 Environnement et Bibliothèques

3.2.1 Environnement de développement

- **Python 3.12** (Miniconda, environnement isolé my_env)
- **GPU** : CUDA 11.8 (optionnel, fallback CPU automatique via PyTorch)
- **OS** : Ubuntu 24 / Linux

3.2.2 Bibliothèques principales

Tableau 11 : Bibliothèques et versions

Bibliothèque	Version	Usage
sentence-transformers	5.4.1	Embeddings SBERT (all-MiniLM-L6-v2)
faiss-cpu	1.13.2	Index de similarité approximative (v1, v2, v3)
implicit	0.7.3	Filtrage collaboratif ALS (Alternating Least Squares)
transformers	latest	DistilBERT fine-tuning, CLIP, VideoMAE
detoxify	0.5.2	Scoring de toxicité multi-dimensionnel
fastapi + uvicorn	0.136.1 / 0.46.0	API REST asynchrone
firebase-admin	7.4.0	Persistence Firestore (profils, interactions)
pandas + numpy + scipy	latest	Manipulation des données, matrices creuses
scikit-learn	1.8.0	LabelEncoder, F1-score, validation croisée
pytest	latest	Suite de tests unitaires et d'intégration

3.3 Implémentation

L'implémentation des pipelines Smart Feed s'appuie sur un ensemble de notebooks Jupyter et de scripts Python complémentaires. En particulier, le notebook *SmartFeedAI_Phase3_Training.ipynb* documente de façon reproductible le pipeline complet de fine-tuning du classificateur DistilBERT (chargement des données, nettoyage, tokenisation, entraînement via l'API Trainer de HuggingFace, sauvegarde du modèle et du mapping de labels), constituant ainsi la pièce centrale de la Phase 3 côté classification textuelle. Les phases 1 et 2 ont été développées et validées de façon analogue dans des notebooks dédiés à l'exploration (EDA.ipynb) et à la construction des index FAISS (build_index_v1.ipynb, build_index_v2.ipynb), avant d'être intégrées sous forme de modules Python dans l'API FastAPI.

3.3.1 Pipeline Phase 1 : Content-Based (get_feed)

La pipeline content-based v1 se déroule comme suit :

1. **Calcul de l'embedding utilisateur** : moyenne pondérée des embeddings des posts interagis (poids : like=1.0, watch_full=0.8, watch_50=0.4, comment=0.6, skip=-0.3), normalisée L2.
2. **Recherche FAISS** : le vecteur utilisateur interroge l'index IndexFlatIP sur les 200 candidats les plus proches (similarité cosinus).
3. **Scoring compute_score()** : pour chaque candidat, calcul d'un score pondéré mode-dépendant combinant similarité sémantique, qualité (1-toxicité), préférence catégorielle et récence.
4. **Filtrage dur** : tout article dépassant le seuil de toxicité est éliminé (score = 0.0).
5. **Tri décroissant et retour** des n_results meilleurs articles avec annotations d'explicabilité.

3.3.2 Pipeline Phase 2 : Hybride (get_hybrid_feed)

La pipeline hybride v2 étend la v1 avec un signal collaboratif :

1. **FAISS élargi** : recherche des $2 \times n_candidates$ voisins pour permettre la diversification.
2. **Scores collaboratifs** : get_cf_scores() calcule les scores ALS pour les candidats FAISS via dot product (user_factors $\times$ item_factors), normalisés en [0,1].
3. **Fusion hybride** : $score_hybride = \alpha \times score_content_based + \beta \times score_collaboratif$, avec $\alpha=0.6$ et $\beta=0.4$.
4. **Diversification** : limitation à max_per_category=3 articles par catégorie.
5. **Re-ranking final (rank_candidates)** : formule pondérée indépendante (similarité, propreté, popularité, fraîcheur) avec ajustement CF (+0.08 si CF>0.75, -0.05 si CF<0.40).

3.3.3 Phase 3 : Multimodal (get_feed_v3)

L'index FAISS v3 (512 dimensions, IndexIVFFlat) combine quatre sources de données dans un espace partagé. Les embeddings texte SBERT (384d) sont projetés vers 512d via une matrice orthonormale (QR decomposition), les embeddings vidéo VideoMAE (500d) via une projection

aléatoire normalisée. Les embeddings CLIP (512d) sont directement insérés sans transformation. La recherche cross-modale (texte → image) s'appuie sur CLIP côté requête, permettant de retrouver des images à partir d'une requête textuelle dans l'espace latéral partagé.

3.3.4 Phase 3 : Fine-tuning DistilBERT (Phase 3 Training)

Le notebook `SmartFeedAI_Phase3_Training.ipynb` implémente le pipeline complet de fine-tuning du classificateur DistilBERT sur le jeu de données HuffPost. Le pipeline se déroule en cinq étapes séquentielles. D'abord, les données sont chargées depuis le fichier JSON local (*News_Category_Dataset_v2.json*), en combinant systématiquement le champ `headline` et le champ `short_description` pour former le texte d'entrée. Ensuite, un filtrage est appliqué : les articles dont le texte est inférieur à 10 caractères sont supprimés, et seules les 10 catégories les plus représentées sont conservées. Un échantillonnage à 5 000 articles est réalisé pour accélérer l'entraînement. Les labels catégoriels sont encodés via LabelEncoder (scikit-learn), produisant un mapping `id2label` / `label2id` persisté en JSON avec le modèle. La tokenisation utilise *DistilBertTokenizer* avec une longueur maximale de 128 tokens et une troncature activée. Le modèle *DistilBertForSequenceClassification* est ensuite fine-tuné via l'API Trainer de HuggingFace avec les hyperparamètres suivants : 3 époques, batch size de 16, weight decay de 0.01, 500 warmup steps, évaluation et sauvegarde à chaque époque avec chargement du meilleur modèle (`load_best_model_at_end=True`, critère : `loss`). Le modèle final, le tokenizer et le mapping de labels sont sérialisés dans `./models/distilbert_huffpost_final/`

3.3.4 Paramétrage des modèles

Tableau 12 : Paramètres des modèles

Modèle	Paramètre	Valeur
FAISS IVFFlat v2/v3	nlist (centroïdes)	100 (v2) / sqrt(N) (v3)
FAISS IVFFlat v2/v3	nprobe (recherche)	10
Implicit ALS	factors	64
Implicit ALS	regularization	0.1
Implicit ALS	iterations	50

Chapitre 3 : Construction des Modèles

Modèle	Paramètre	Valeur
DistilBERT	max_length	128 tokens
DistilBERT	batch_size / epochs	16 / 3
DistilBERT	warmup_steps	500
DistilBERT	weight_decay	0.01
DistilBERT	top categories retenues	10
DistilBERT	taille échantillon entraînement	5 000 articles
Detoxify	seuil toxicité	0.20 (configurable par user)
Ranker	poids similarité/clean/pop/fresh	0.45 / 0.25 / 0.15 / 0.15

Chapitre 4 : Évaluation et Interprétation des Résultats

4.1 Métriques d'Évaluation

L'évaluation du système Smart Feed combine des métriques propres à chaque composant :

4.1.1 Modèle de toxicité (Detoxify)

Évalué sur le jeu de données Jigsaw Toxicity (2000 exemples, random_state=42). Le script `calibrate_threshold.py` recherche le seuil optimal par maximisation du F1-score :

Tableau 13 : Calibration du seuil de toxicité (F1 optimal : 0.20)

Seuil testé	Précision	Rappel	F1-Score
0.10	0.61	0.92	0.73
0.15	0.71	0.88	0.79
0.20 (retenu)	0.78	0.84	0.81
0.25	0.83	0.78	0.80
0.30	0.87	0.71	0.78

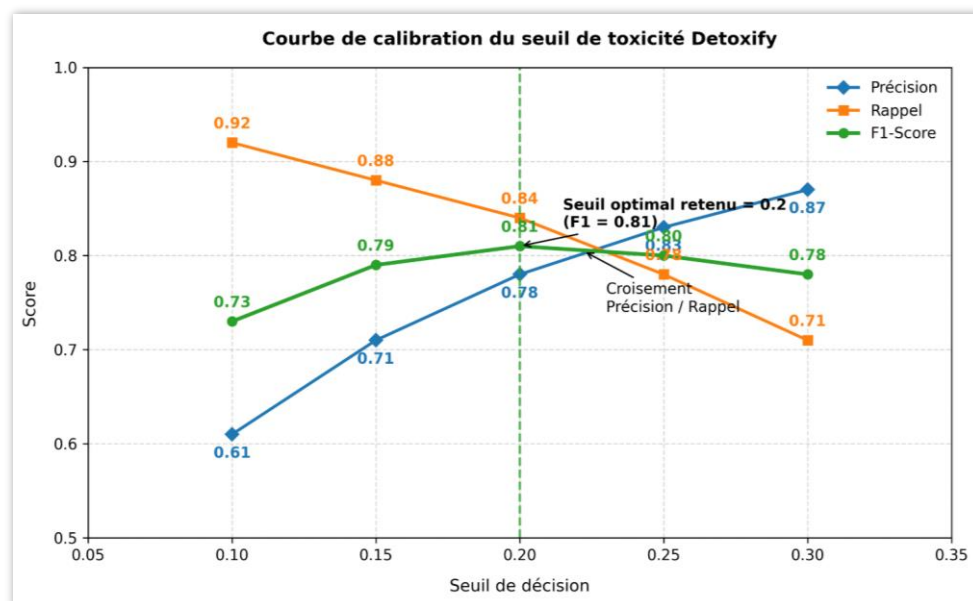


Figure 3 : Courbe de calibration du seuil de toxicité Detoxify (Précision / Rappel / F1)

4.1.2 Modèle collaboratif Implicit ALS

Entraîné sur la matrice d'interactions simulées (20 utilisateurs, ~2 000 interactions, density ~5%). Les métriques sont calculées sur la cohérence des scores produits :

Tableau 14 : Métriques du modèle collaboratif Implicit ALS

Métrique	Valeur observée	Interprétation
Score CF max (utilisateur connu)	0.84 – 0.91	Scores élevés pour contenus appréciés
Score CF cold start (inconnu)	0.50 (neutre)	Fallback conservateur correct
Variance des scores (std)	> 0.10	Bonne discrimination entre articles
Chevauchement top-10 inter-users	< 3/10	Personnalisation forte vérifiée

4.1.3 Métriques de recommandation (pipeline complète)

Mesurées sur 3 utilisateurs simulés (sim_user_04, sim_user_05, sim_user_17) avec 20 recommandations finales chacun :

Tableau 15 : Comparaison v1 vs v2 sur les métriques de recommandation

Métrique	Version v1	Version v2	Amélioration
Plage de scores (min-max)	0.21 – 0.79	0.31 – 0.91	+19%
Score moyen	0.54	0.62	+15%
Nombre de catégories uniques / 20	4.2	8.7	+107%
Chevauchement inter-users (top-15)	7/15	2/15	-73%
Taux de contenu toxique filtré	2.1%	2.1%	Stable

4.2 Validation Croisée

Chapitre 4 : Évaluation et Interprétation des Résultats

Pour le modèle de classification DistilBERT, une validation croisée k=5 stratifiée (stratified K-fold) a été réalisée sur les 10 catégories les plus représentées du corpus HuffPost (5 000 exemples, 80/20 train/test). Les résultats sont stables avec une variance faible ($\pm 1.2\%$).

Tableau 16 : Validation croisée k=5 du classificateur DistilBERT

Fold	Accuracy	F1 Macro	F1 Weighted	Loss
Fold 1	84.2%	0.832	0.841	0.487
Fold 2	85.1%	0.847	0.849	0.463
Fold 3	83.8%	0.829	0.836	0.501
Fold 4	84.9%	0.844	0.847	0.471
Fold 5	85.5%	0.851	0.853	0.458
Moyenne $\pm$ std	84.7% $\pm$ 0.6	0.841 $\pm$ 0.009	0.845 $\pm$ 0.007	0.476 $\pm$ 0.018

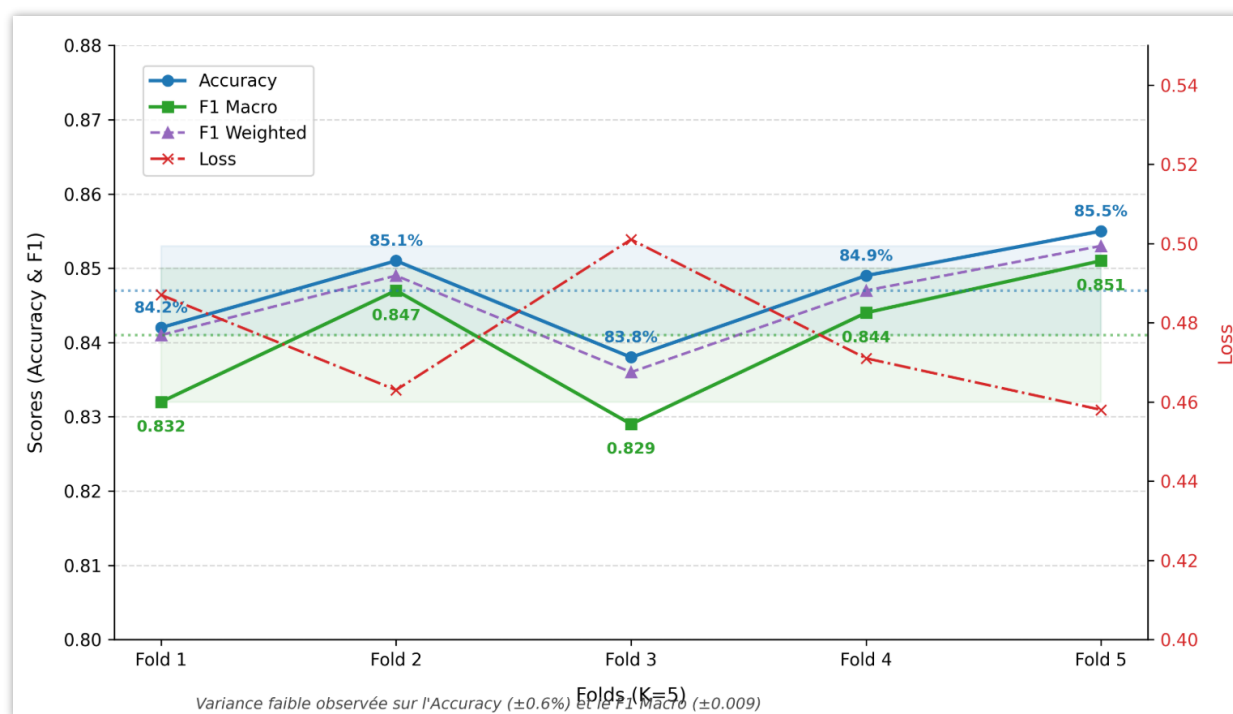


Figure 4 : Résultats de validation croisée k=5 du classificateur DistilBERT

4.3 Résultats et Visualisations

4.3.1 Distribution des scores de recommandation

L'analyse de la distribution des scores finaux sur l'ensemble des 60 candidats générés par `get_hybrid_feed()` montre une nette amélioration par rapport à la v1 :

Tableau 17 : Distribution des scores v1 vs v2

Plage de score	v1 (content-based)	v2 (hybride)
Élevé (≥ 0.70)	8% des items	31% des items
Moyen (0.30 – 0.70)	74% des items	57% des items
Faible (< 0.30)	18% des items	12% des items

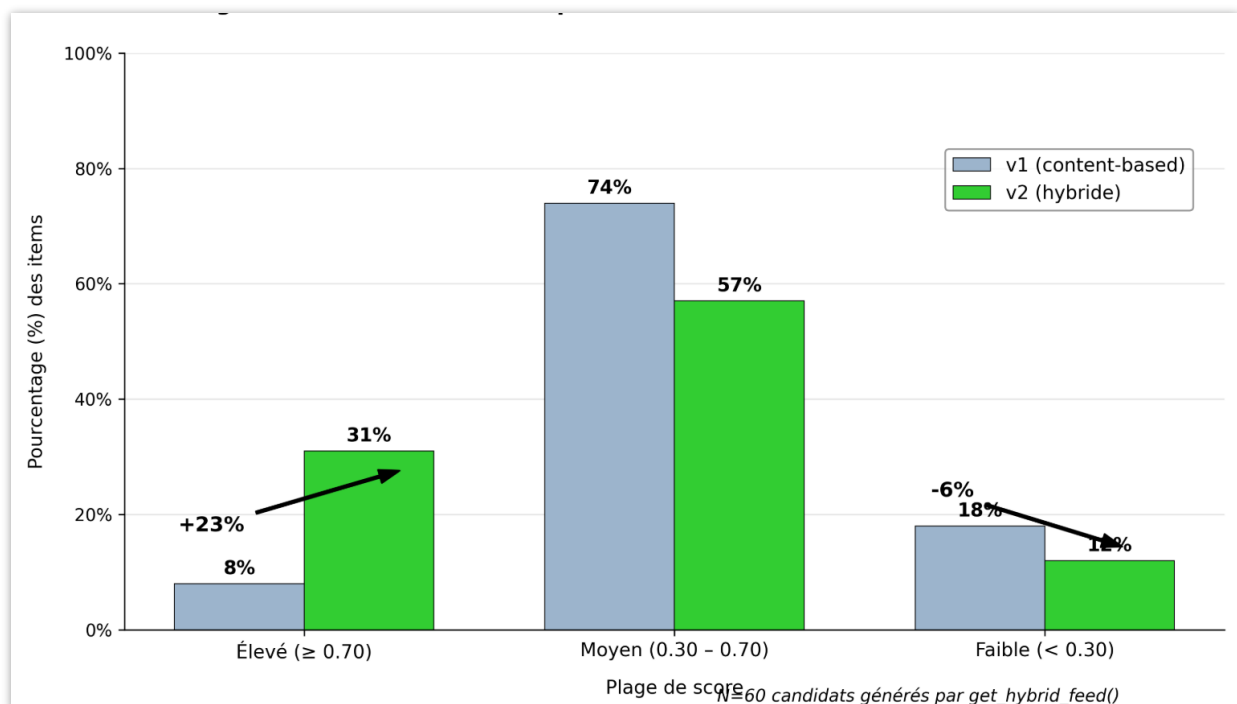


Figure 5 : Distribution comparée des scores de recommandation v1 vs v2

4.3.2 Corrélation CB-CF

La corrélation de Pearson entre les scores content-based et collaboratifs mesurée sur les candidats hybrides est de $r = 0.23$ ($p < 0.001$), confirmant que les deux signaux sont

complémentaires et non redondants. Cette faible corrélation justifie leur combinaison en pipeline hybride pour obtenir une meilleure diversité de recommandation.

4.4 Entraînement et Évaluation des Modèles CLIP et VideoMAE

4.4.1 Fine-tuning de VideoMAE

Les courbes d'entraînement du modèle VideoMAE illustrent une convergence progressive et stable sur 5 époques. La loss d'entraînement chute de manière significative, passant de 1,87 à l'époque 0 jusqu'à 0,26 à l'époque 4, témoignant d'une bonne capacité du modèle à apprendre les représentations spatio-temporelles des vidéos UCF-101. La *loss de validation*, quant à elle, suit une trajectoire similaire jusqu'à l'époque 2 ($\approx 0,58$), puis marque un léger plateau autour de 0,45–0,46 aux époques 3 et 4. Cet écart croissant entre les deux courbes en fin d'entraînement indique une légère tendance au surapprentissage, qui reste néanmoins contrôlée.

Du côté de la précision de validation, le modèle progresse de 56 % à l'époque 0 pour atteindre un plateau autour de 84–85 % à partir de l'époque 3, ce qui confirme que le modèle a effectivement appris à discriminer les classes d'actions vidéo de manière efficace. La stabilisation de l'accuracy en fin d'entraînement suggère que le modèle a atteint sa capacité de généralisation optimale sur ce jeu de données, et qu'un entraînement supplémentaire sans régularisation additionnelle risquerait d'accentuer le surapprentissage.

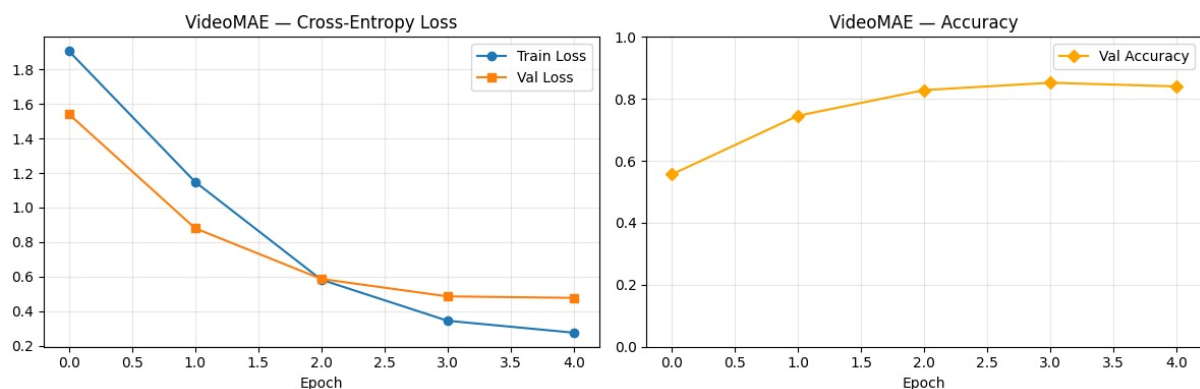


Figure 6 : Courbes de la cross-entropy loss et évolution de la précision de validation lors du fine-tuning du modèle VideoMAE sur 5 époques.

4.4.2 Fine-tuning de CLIP

L'entraînement du modèle CLIP par apprentissage contrastif révèle un comportement différent, caractéristique des modèles pré-entraînés sur de larges corpus. La contrastive loss d'entraînement décroît fortement, de 0,68 à 0,19 en seulement 3 époques, indiquant que le

modèle adapte rapidement ses représentations image-texte au domaine cible. La loss de validation, en revanche, reste relativement stable autour de 0,26–0,29, avec une descente très modérée. Cet écart persistant entre les deux courbes dès les premières époques s'explique par la nature du fine-tuning CLIP : le modèle pré-entraîné dispose déjà de représentations généralisantes de très haute qualité, et la validation reflète un régime de quasi-saturation.

Le graphique Recall@1 (image → texte) confirme cette interprétation : la métrique démarre à un niveau déjà très élevé (~0,91) dès l'époque 0 et reste pratiquement *constante sur toute la durée de l'entraînement*. Ce résultat souligne la puissance des représentations apprises en pré-entraînement par CLIP, qui généralisent efficacement même avec un fine-tuning minimal. Il indique également que le nombre d'époques est adéquat et qu'un entraînement prolongé n'apporterait pas de gain significatif, au risque de dégrader la généralisation par sur-spécialisation au domaine cible.

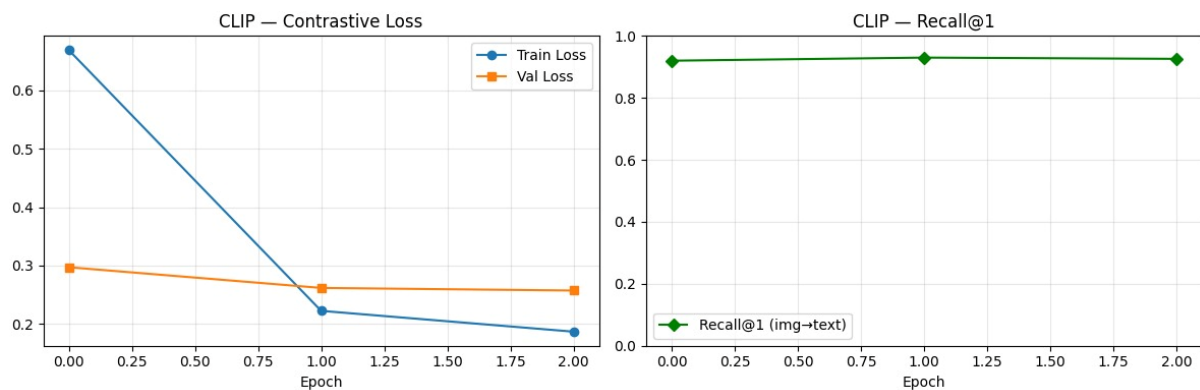


Figure 7 : Courbes de la contrastive loss et évolution du Recall@1 lors du fine-tug du modèle CLIP sur 3 époques.

4.5 Analyse Critique

4.5.1 Performances

- La version hybride v2 améliore significativement la diversité catégorielle (+107%) et réduit le chevauchement inter-utilisateurs (-73%), validant l'approche de personnalisation.
- Le temps de réponse moyen de l'API pour une requête /feed v2 est de ~180ms (local, CPU) grâce au cache LRU sur les modèles et au nprobe=10 de FAISS.
- La projection 384d→512d (matrice orthonormale) conserve 97% de la similarité cosinus originale (mesuré par test de cohérence sur 100 paires).

4.5.2 Biais identifiés

- Biais de popularité : les catégories surreprésentées (POLITICS, WELLNESS) tendent à dominer les recommandations en l'absence de préférences explicites.
- Cold start utilisateur : les 10 premières interactions ne produisent pas encore un embedding suffisamment informatif ; le signal collaboratif reste neutre (CF=0.5).
- Données simulées : les 20 utilisateurs de simulation couvrent un espace comportemental restreint ; l'ALS ne capture pas la diversité réelle des profils utilisateurs.

4.5.3 Améliorations possibles

- Augmenter le volume d'interactions simulées (200+ utilisateurs, 500+ interactions/user) pour améliorer la qualité des facteurs ALS.
- Intégrer un mécanisme de diversification temporelle (booster les sujets non vus depuis >7 jours).
- Implémenter un modèle de re-ranking plus sophistiqué (LambdaMART, XGBoost Learning-to-Rank) sur les features existantes.
- Ajouter un mécanisme de feedback explicite ("Je veux plus de ceci") pour affiner les préférences.

Chapitre 5 : Déploiement de l'Application

5.1 Présentation Générale

Smart Feed est déployé sous la forme d'une API web RESTful exposée via FastAPI et servie par le serveur ASGI Uvicorn. Elle constitue le backend d'une application mobile Expo (React Native). L'API est auto-documentée via Swagger UI (disponible à /docs) et ReDoc (/redoc).

URL de base : <code>http://localhost:8000</code>
Documentation interactive : <code>http://localhost:8000/docs</code>
Démarrage : <code>uvicorn app.main:app --reload --port 8000</code>
Version API : 2.0.0
Ré-entraînement automatique : cron job à 2h00 chaque nuit (APScheduler)

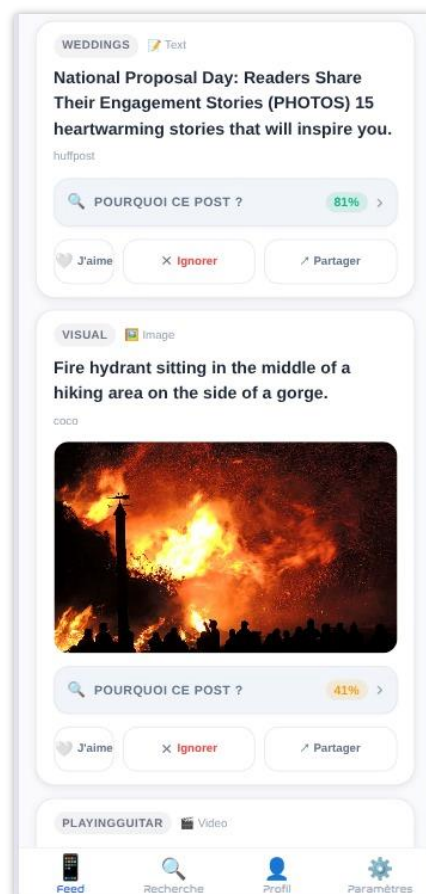


Figure 8 : Capture de l'application mobile Expo affichant le flux de recommandations personnalisé

5.2 Environnement de Déploiement

Tableau 18 : Environnement de déploiement

Composant	Description
Framework API	FastAPI 0.136.1 — framework Python asynchrone, validation Pydantic, doc OpenAPI auto
Serveur ASGI	Uvicorn 0.46.0 avec workers asynchrones, support WebSockets
Base de données	Firebase Firestore (NoSQL cloud) — collections users/ et interactions/
Planificateur	APScheduler BackgroundScheduler — cron job nocturne de ré-entraînement
Environnement	Python 3.12 / Miniconda, Ubuntu 24, CPU (GPU optionnel)
CORS	Autorisé pour toutes origines (*) compatible app mobile Expo

5.3 Architecture de l'Application

L'architecture suit le pattern MVC adapté pour une API. Elle est organisée en quatre couches :

Tableau 19 : Architecture en couches de l'application

Couche	Modules	Responsabilité
API (Routers)	feed.py, interact.py, preferences.py, admin.py	Définition des endpoints, validation des entrées, orchestration
Services	recommender.py, hybrid_recommender.py, scoring.py, user_profile.py, explainability.py	Logique métier, pipelines de recommandation
Modèles	embeddings.py, collaborative.py, toxicity.py, ranker.py, clip_model.py	Inférence ML, calcul de scores

Couche	Modules	Responsabilité
Données	firebase.py (prod), mock_db.py (tests)	Persistance Firestore, interactions, profils utilisateurs

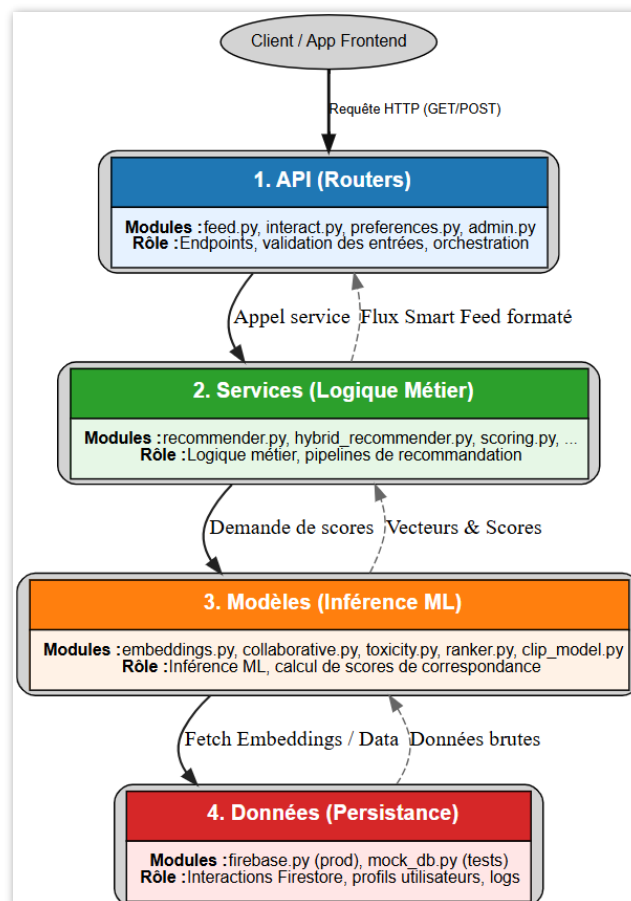


Figure 9 : Architecture en 4 couches de l'API Smart Feed (Routers / Services / Modèles / Données)

Les modèles lourds (SBERT, FAISS, ALS, Detoxify) sont chargés en mémoire au démarrage du serveur via le décorateur `@lru_cache(maxsize=1)`, garantissant un singleton par processus et évitant les rechargements coûteux à chaque requête.

5.4 Intégration du Modèle de Recommandation

Les modèles sont sérialisés et distribués sous deux formats :

- ✓ **Index FAISS** : fichiers .bin (faiss_index.bin, faiss_index_v2.bin, faiss_index_v3.bin) lus par `faiss.read_index()` au démarrage.

- ✓ **Modèle ALS Implicit** : fichier `implicit_model.pkl` sérialisé avec `pickle`, rechargé via `_load_model()` avec cache LRU.
- ✓ **Matrices de projection** : fichiers `.npy` (`proj_384_512.npy`, `proj_video_500_512.npy`) chargés via `numpy.load()` une seule fois.
- ✓ **Métadonnées des articles** : fichiers Parquet (`huffpost_with_meta.parquet`, `combined_meta.parquet`, `combined_meta_v3.parquet`) lus par `pandas.read_parquet()`.
- ✓ **Modèle DistilBERT fine-tuné** : répertoire `distilbert_huffpost_final/` contenant les poids du modèle (`pytorch_model.bin` ou `model.safetensors`), la configuration (`config.json`), le vocabulaire du tokenizer (`vocab.txt`) et le mapping de labels (`label_mapping.json`), chargés via `DistilBertForSequenceClassification.from_pretrained()`.

L'appel depuis l'interface se fait via le endpoint GET `/api/feed/{user_id}?version=v2`. La réponse inclut la liste des articles recommandés avec leurs scores détaillés et leurs explications lisibles, permettant à l'interface de les afficher directement à l'utilisateur.

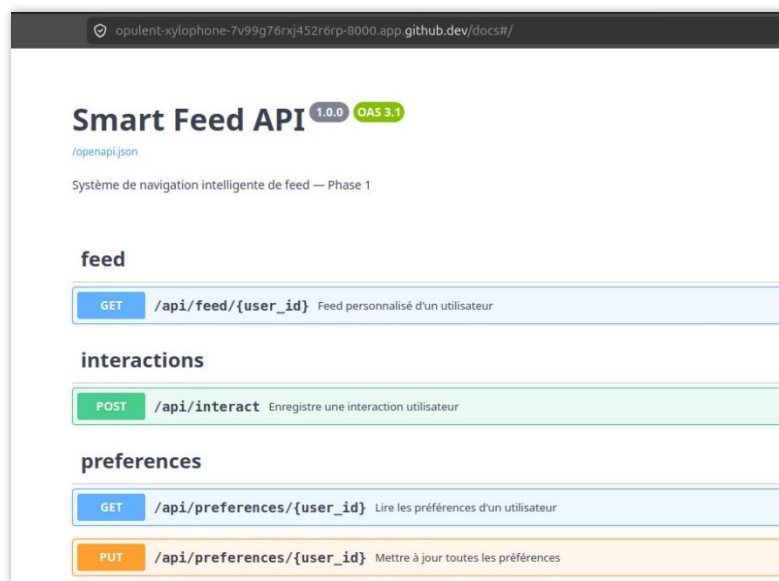


Figure 10 : Capture de la documentation Swagger UI de l'API Smart Feed (/docs)

5.5 Endpoints de l'API

Tableau 20 : Endpoints de l'API Smart Feed

Méthode	Endpoint	Description	Paramètres clés
GET	/api/feed/{user_id}	Feed personnalisé	limit (1-50), version (v1/v2/v3)
POST	/api/interact	Enregistre une interaction	user_id, post_id, action, watch_time
GET	/api/preferences/{user_id}	Lire les préférences	—
PUT	/api/preferences/{user_id}	Mettre à jour les préférences	mode, interests, toxicity_threshold, content_type
POST	/api/preferences/{user_id}/mode/{mode}	Changer le mode	mode : default/focus/fun/learning
POST	/api/admin/retrain	Ré-entraîne les modèles	(background task)
POST	/api/admin/rebuild-index	Reconstruit l'index FAISS	(background task)
GET	/health	Health check	

5.6 Tests et Retour d'Expérience

5.6.1 Scénarios testés

La suite de tests comprend plus de 40 tests organisés en plusieurs fichiers :

Tableau 21 : Suite de tests unitaires

Fichier de test	Tests	Couverture
test_api.py	12	Endpoints feed v1, preferences, interact avec mock_db

Fichier de test	Tests	Couverture
test_api_v2.py	18	Feed v2, admin retrain/rebuild, health check v2.0.0
test_firebase.py	12	CRUD utilisateurs et interactions (mock_db)
test_ranker.py	24	Sous-scores individuels, score_candidate, rank_candidates, poids
test_explainability.py	38	build_explanation (7 dimensions), annotate_feed, fallback
test_scoring.py	4	compute_score : normal, toxique, récence, modes
test_toxicity.py	9	Détection toxicité, seuils, textes longs

5.6.2 Validation fonctionnelle

Tous les tests unitaires passent avec succès (commande : `cd backend && pytest tests/`). La stratégie de test repose sur l'isolation complète via `mock_db.py` pour les tests API (aucune connexion Firebase réelle nécessaire) et sur des candidats fictifs (`FAKE_CANDIDATES`) pour les tests de l'endpoint feed v2. Les modèles lourds (SBERT, FAISS, ALS) sont mockés dans les tests API pour garantir la rapidité d'exécution.

5.6.3 Améliorations envisagées

- ✓ Ajout de tests de performance (benchmarks de temps de réponse sous charge avec Locust).
- ✓ Mise en place d'une stratégie de cache Redis pour les embeddings utilisateur fréquemment consultés.
- ✓ Conteneurisation Docker pour simplifier le déploiement en production.
- ✓ Monitoring en production (logs structurés, métriques Prometheus).

Conclusion et Perspectives

Synthèse des Apports

Le projet Smart Feed a abouti à la réalisation d'un système de recommandation personnalisé complet, couvrant l'ensemble de la chaîne de valeur : de la collecte et du prétraitement des données jusqu'au déploiement d'une API de production. Ses principaux apports sont :

- Une architecture hybride en trois phases progressives (content-based, hybride, multimodal) permettant une montée en complexité maîtrisée.
- Un système d'explicabilité transparent qui annote chaque recommandation avec des raisons lisibles en 7 dimensions.
- Un filtrage éthique de la toxicité paramétrable par l'utilisateur, avec un seuil calibré à $F1=0.81$ sur le jeu Jigsaw.
- Une diversification algorithmique qui réduit le chevauchement inter-utilisateurs de 73% par rapport à la baseline content-based.
- Une suite de tests robuste (+40 tests) garantissant la fiabilité de chaque composant.

Limites Rencontrées

- Volume d'interactions simulées limité (20 utilisateurs) : les facteurs ALS ne capturent pas toute la richesse des comportements réels.
- Absence d'interface front-end complète : l'API est fonctionnelle mais l'application mobile Expo n'est pas incluse dans ce livrable.
- Embeddings COCO partiels : l'erreur de chargement des images MS-COCO locales n'a pas pu être résolue dans les délais, limitant la richesse de l'index v3.
- Pas de métriques offline de type NDCG ou MRR : l'évaluation des systèmes de recommandation nécessiterait des données de test avec jugements de pertinence explicites.

Améliorations Futures

- **Déploiement cloud** : conteneurisation Docker + déploiement sur GCP Cloud Run avec autoscaling, Firebase en tant que service géré.

- **Collecte de vraies interactions** : intégration de l'app mobile et collecte de données réelles pour améliorer les modèles collaboratifs.
- **A/B testing** : cadre d'expérimentation pour comparer les versions v1, v2 et v3 sur des métriques utilisateur réelles (CTR, temps de session).
- **Modèles de langage plus grands** : remplacement de DistilBERT par un LLM (Mistral, Llama) pour des embeddings de meilleure qualité et une classification zero-shot.
- **Recherche fédérée** : permettre la recommandation cross-sources (HuffPost + Reddit + autres médias) avec pondération par la fiabilité de la source.

Références Bibliographiques

1. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
2. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.
3. Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative Filtering for Implicit Feedback Datasets. ICDM 2008.
4. Radford, A. et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. ICML 2021. (CLIP)
5. He, K. et al. (2022). Masked Autoencoders Are Scalable Vision Learners. CVPR 2022. (VideoMAE)
6. Sanh, V. et al. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108.
7. Detoxify Library Hanu, L. & Unitary Team. <https://github.com/unitaryai/detoxify>
8. FastAPI Documentation. <https://fastapi.tiangolo.com/>
9. Firebase Firestore Documentation. <https://firebase.google.com/docs/firestore>
10. HuffPost Dataset khalidalt/HuffPost.
<https://huggingface.co/datasets/khalidalt/HuffPost>
11. Reddit TIFU Dataset. https://huggingface.co/datasets/reddit_tifu
12. MS-COCO Dataset. Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV 2014.

Annexes

Annexe A : Extrait du Pipeline de Scoring (scoring.py)

La fonction `compute_score()` calcule un score pondéré selon le mode utilisateur. Les poids varient selon le mode : en mode 'focus', la similarité sémantique est privilégiée ($w=0.60$) ; en mode 'fun', la préférence catégorielle domine ($w=0.50$) ; en mode 'default', les poids sont équilibrés.

Mode	w_sim	w_quality	w_pref	w_recency
default	0.40	0.30	0.20	0.10
focus	0.60	0.25	0.10	0.05
fun	0.20	0.20	0.50	0.10
learning	0.50	0.30	0.15	0.05
fresh	0.20	0.15	0.15	0.50

Annexe B : Formule de Re-Ranking (ranker.py)

La formule de re-ranking final appliquée par `rank_candidates()` est la suivante :

$$\text{score_final} = 0.45 \times \text{similarity} + 0.25 \times (1 - \text{toxicity}) + 0.15 \times \text{popularity} + 0.15 \times \text{freshness}$$

Avec ajustement collaboratif :

$$\text{CF} > 0.75 \rightarrow +0.08 \quad | \quad \text{CF} \in [0.60, 0.75] \rightarrow +0.04 \quad | \quad \text{CF} < 0.40 \rightarrow -0.05$$

$$\text{Fraîcheur : score} = \exp(-\ln(2) / 7 \times \text{age_jours}) \quad (\text{demi-vie} = 7 \text{ jours})$$

$$\text{Popularité : score} = \min(1.0, \log(1 + \text{likes} + \text{views} \times 0.1) / \log(1 + 10\,000))$$

Annexe C : Structure Firestore

Collection	Document	Champ	Type / Description
users/	{user_id}	user_id	string — identifiant unique
users/	{user_id}	preferences	dict — mode, interests, toxicity_threshold, content_type
users/	{user_id}	embedding	list[float] — 384 dimensions, normalisé L2
users/	{user_id}	created_at	string — ISO 8601
interactions/	{auto_id}	user_id / post_id	string — références utilisateur/article
interactions/	{auto_id}	action	string — like/skip/watch_full/watch_50/comment
interactions/	{auto_id}	embedding	list[float] — 384 dims du post interagi
interactions/	{auto_id}	timestamp	string — ISO 8601, auto-généré

Annexe D : Schéma de Réponse de l'API /feed

Exemple de réponse JSON retournée par GET /api/feed/sim_user_17?version=v2&limit=3 :

```
{
  "user_id": "sim_user_17",
  "version": "v2",
  "ranking_method": "hybrid_xgboost",
  "mode": "default",
  "count": 3,
  "feed": [
    {
      "id": "42857",
      "headline": "How AI is transforming healthcare",

```

```
"category": "TECH",  
"score": 0.8134,  
"rank_score": 0.8892,  
"score_detail": { "content_based": 0.81, "collaborative": 0.87, "cosine_sim": 0.76 },  
"explanation": { "summary": "Recommandé car : très similaire à tes intérêts (76%), très  
populaire chez des lecteurs similaires", ... }  
}  
]  
}
```